Functional and Non-Functional Requirements

1. Introduction

System Name: SyncSpace — Real-Time Collaborative Code Editor

Purpose: SyncSpace enables multiple developers to simultaneously edit the same code document in real time through a web-based editor. It uses Conflict-free Replicated Data Types (CRDTs) and WebSocket communication to ensure all participants see a consistent document state without a central authority dictating edit order.

Scope: The system comprises a web-based code editor frontend (using an open-source editor component such as CodeMirror or Monaco), a WebSocket relay server for real-time message routing, and a CRDT engine (Yjs) for automatic conflict resolution. The prototype targets small collaborative sessions (up to 3 concurrent users).

---

2. Functional Requirements

2.1 Session Management

ID	Name	Description	Priority	Architecturally Significant
FR-SM-01	Create Session	A user shall be able to create a new collaboration session. The system generates a unique session ID; each **client** initializes a **Yjs** shared document in the browser, and the relay only routes session-scoped **WebSocket** traffic (it does not hold the CRDT on the server).	Must	Yes — establishes the CRDT document lifecycle and WebSocket channel setup.
FR-SM-02	Join Session	A user shall join an existing session via a shared session link/ID. Upon joining, the client receives the full current document state within ≤ 3 seconds.	Must	Yes — drives the state synchronization protocol design (full-state snapshot transfer).
FR-SM-03	Leave Session	When a user disconnects (intentionally or due to network drop), the system detects the disconnection within ≤ 5 seconds via heartbeat timeout.	Must	No

2.2 Real-Time Co-Editing

ID	Name	Description	Priority	Architecturally Significant
FR-CE-01	Local-First Editing	Every keystroke shall be immediately applied to the local CRDT replica so the user	Must	Yes — this is the core "local-first" architectural decision; edits are never blocked by the network.

		experiences zero perceivable input delay.		
FR-CE-02	Remote Sync	Local edit operations shall be broadcast to all connected peers via the relay server. Remote peers shall see the changes within ≤ 1 second under normal network conditions.	Must	Yes — determines the WebSocket message protocol and CRDT update vector format.
FR-CE-03	Conflict Resolution	When two or more users edit the same region concurrently, the CRDT merge function shall deterministically resolve the conflict so all replicas converge to an identical state.	Must	Yes — this is the single most architecturally significant requirement; it justifies the choice of CRDT over OT.
FR-CE-04	Syntax Highlighting	The editor shall provide syntax highlighting for common programming languages (at minimum: Java, Python, C++).	Should	No

2.3 Presence Awareness

ID	Name	Description	Priority	Architecturally Significant
FR-PA-01	Live Cursors	Each remote user's cursor position shall be displayed in the editor with a unique color and name label.	Must	Yes — requires a separate "awareness" data channel alongside the document CRDT, using Yjs Awareness protocol.
FR-PA-02	Selection Highlight	Remote user text selections shall be highlighted with a translucent overlay of the user's assigned color.	Should	No
FR-PA-03	Participant List	The UI shall display a list of all currently connected participants with their names/colors.	Should	No

2.4 User Interface

ID	Name	Description	Priority	Architecturally Significant
FR-UI-01	Code Editor	The system shall provide a full-featured code editor (based on CodeMirror or Monaco) with line numbers, auto-indentation, and bracket matching.	Must	No
FR-UI-	Session Controls	The UI shall provide controls to create a new session and to join an existing session	Must	No

02		by entering a session ID.		
FR-UI-03	Connection Status	The UI shall display the current connection status (connected / reconnecting / disconnected).	Should	No

---

3. Non-Functional Requirements

3.1 Performance

ID	Metric	Target	Threshold	Architecturally Significant
NFR-P-01	End-to-end edit sync latency	≤ 500 ms	≤ 1 second	Yes — drives the choice of WebSocket over HTTP polling, and the use of incremental CRDT update vectors instead of full-state transfers.
NFR-P-02	Local keystroke-to-render latency	≤ 16 ms	≤ 50 ms	Yes — requires local-first architecture where edits apply to the local replica before network round-trip.
NFR-P-03	State convergence time (after final edit)	≤ 1 second	≤ 2 seconds	Yes — validated by CRDT's mathematical convergence guarantee.

Why these are architecturally significant: The latency and convergence requirements together mandate the **local-first CRDT architecture** — edits cannot wait for a server round-trip (ruling out centralized OT), and convergence must be guaranteed mathematically (ruling out ad-hoc merge logic). This is the fundamental architectural driver of the entire system.

3.2 Concurrency / Scalability

ID	Metric	Target	Threshold	Architecturally Significant
NFR-SC-01	Concurrent users per session	3	2	Yes — affects relay server fan-out design and CRDT state vector size.
NFR-SC-02	Maximum document size with acceptable performance	100 KB	50 KB	No

Constraint rationale: These limits reflect the scope of a semester course project, not empirically derived performance ceilings. The prototype is designed to demonstrate architectural concepts (CRDT-based collaboration, stateless relay) rather than to serve production workloads.

3.3 Reliability / Fault Tolerance

ID	Metric	Target	Architecturally Significant
NFR-R-01	Data convergence	100% (guaranteed)	Yes — this is the core CRDT invariant: all replicas that receive the same set of updates will reach an identical **syntactic** state. Note: convergence does not

	after network partition heals		guarantee **semantic** correctness — concurrent edits to overlapping regions may merge into unintended output (e.g. interleaved code blocks), requiring user review.
NFR-R-02	Client reconnection and resync time	≤ 5 seconds	Yes — requires the relay to track active WebSocket connections per session, and uses **peer-driven state exchange** (`request_state` → peer replies with `encodeStateAsUpdate`) for incremental resync. The relay itself does not hold Yjs document state.
NFR-R-03	Offline edit buffering	≥ 100 operations	No — **Prototype caveat:** the current fault-tolerance mechanism uses a destructive page reload on WebSocket disconnect (see System.md § Communication Subsystem), which destroys the local `Y.Doc` and any buffered operations. This NFR is therefore **not achievable** in the current prototype; it would require replacing the page reload with a non-destructive reconnect loop.

3.4 Security

ID	Requirement	Priority	Architecturally Significant
NFR-S-01	Session IDs shall be sufficiently random to prevent guessing.	Must	No
NFR-S-02	WebSocket communication shall use WSS (TLS encrypted) in production deployment.	Should	No

3.5 Usability

ID	Requirement	Target
NFR-U-01	Time for a new user to start a collaborative session	≤ 60 seconds
NFR-U-02	The editor shall not break standard keyboard shortcuts	Optional

3.6 Measurement Methodology

To ensure NFR targets are verifiable rather than aspirational, each performance and reliability metric will be measured as follows:

NFR	Measurement Method
NFR-P-01 (E2E sync ≤ 500 ms)	Run the relay server on one device and the benchmark script (`benchmark_nfr.py`) on a separate device on the same LAN. Measure WebSocket round-trip time for CRDT update vectors between two connected clients.
NFR-P-02 (Keystroke latency ≤ 16 ms)	Instrument the client-side Yjs `Y.Doc` transaction with `performance.now()` timestamps before and after local apply. Sample across 100 keystrokes.

NFR-P-03 (Convergence ≤ 1 s)	After a burst of concurrent edits from both clients, compare final document states across replicas and record the wall-clock time until convergence.
NFR-R-02 (Reconnection ≤ 5 s)	Disconnect one client's WebSocket, measure wall-clock time from disconnection event to successful `request_state` completion on rejoin.

3.7 Trust Model

The prototype operates under the following trust assumptions:

- **All connected clients are trusted.** The relay server forwards CRDT and awareness payloads without inspection, parsing, or validation.
- **No input validation on CRDT payloads.** A malicious or buggy client could send malformed Yjs updates, oversized messages, or high-frequency bursts. The relay does not defend against these.
- **Known risks:** malformed updates causing replica divergence; memory exhaustion from oversized messages; denial of service via connection flooding.
- **Scope:** These risks are accepted for the prototype, which targets trusted classroom / LAN environments. A production deployment would add payload-size limits, rate limiting, and schema validation on control messages.

3.8 Documented Trade-offs

The following trade-offs are conscious design choices for the prototype scope:

Trade-off	What we gain	What we accept
Ignoring semantic conflicts	Simpler CRDT merge with no application-level conflict UI	Users must manually review interleaved or nonsensical merged output
Limited scalability (≤ 3 users, ≤ 100 KB)	Relay stays trivially simple; no sharding or load balancing	System is not validated for larger sessions or documents
Simplified fault tolerance (page reload on disconnect)	Minimal client reconnection code	Unsynced local edits are lost on reload; no graceful in-process reconnect
Ephemeral sessions (no persistence)	No database, no backup infrastructure	Document is lost when all clients disconnect
No authentication / authorization	Zero-friction onboarding via capability URLs	No audit trail, no access revocation, no per-user controls

---

4. Architecturally Significant Requirements — Summary

The following requirements are the key architectural drivers for SyncSpace:

Requirement	Why It's Architecturally Significant
FR-CE-01 (Local-First Editing)	Forces a **decentralized, multi-leader replication** architecture where each client is autonomous. No single point of truth for edit ordering.

FR-CE-03 (Conflict Resolution)	Mandates **CRDT** as the synchronization mechanism (over OT), because CRDTs provide mathematical convergence guarantees without a central sequencer.
FR-PA-01 (Live Cursors)	Requires the **Yjs Awareness protocol** — a secondary real-time channel for ephemeral state (cursor positions, user info) separate from the persistent document CRDT.
NFR-P-01 (≤ 1s E2E Latency)	Drives the choice of **WebSocket** transport over HTTP polling, and requires incremental CRDT update encoding (not full-state snapshots on every edit).
NFR-R-01 (100% Convergence)	This is the strongest architectural constraint — it eliminates any "best-effort" sync approach and requires a formally proven merge algorithm (Yjs/CRDT).
NFR-R-02 (Reconnection ≤ 5s)	Requires the relay to track active sessions and uses **peer-driven state exchange** for catch-up on reconnect. The relay forwards a `request_state` message; an existing peer replies with a full or incremental Yjs state update.